

Mini-Projet CY-01

Détection de séquences d'attaque par Automate Fini Déterministe

Mini-SIEM : Corrélation d'événements & Alertes SOC

Table des matières

1	Introduction	3
2	Fondements théoriques : Automate Fini Déterministe	3
2.1	Définition formelle	3
2.2	États et sémantique	3
2.3	Alphabet et correspondance événements	3
2.4	Table de transition δ	4
2.5	Paramètres temporels	4
3	Architecture du workflow n8n	4
3.1	Vue d'ensemble	4
3.2	Description des nœuds	4
3.3	Nœud déclencheur Webhook	5
3.4	Nœuds IF et Switch	5
3.5	Webhook Response	5
4	Persistance des états : Google Sheets SIEM-State-Store	5
4.1	Structure de la feuille	6
4.2	Capture réelle du SIEM-State-Store	6
5	Moteur AFD : Code Node JavaScript	7
5.1	Table de transition implémentée	7
5.2	Logique principale avec fenêtres temporelles	7
6	Système d'alertes multi-canal	8
6.1	Alerte Slack — canal <code>#alerts-siem</code>	8
6.1.1	Configuration technique	8
6.1.2	Payload JSON envoyé	8
6.1.3	Preuve de réception — canal Slack	9
6.2	Alerte WhatsApp — API Twilio Sandbox	9
6.2.1	Présentation de Twilio	9
6.2.2	Activation du sandbox	9
6.2.3	Configuration du nœud HTTP Request (WhatsApp)	9
6.2.4	Réponse API Twilio confirmée	10
6.2.5	Preuve de réception WhatsApp	10
7	Tests et validation	11
7.1	Outil de test : Postman	11
7.2	Séquence d'attaque testée	11
7.3	Résultats étape par étape	12
7.4	Tableau de validation des fonctionnalités	12
8	Difficultés rencontrées et solutions	12
8.1	Persistance d'état entre exécutions	12
8.2	Variabilité du format d'entrée	12
8.3	Authentication Twilio dans n8n	13
8.4	Mode Test vs Production	13

9 Limites identifiées et perspectives	13
10 Conclusion	13

1 Introduction

Ce rapport décrit l'implémentation complète d'un mini-SIEM (Security Information and Event Management) basé sur un Automate Fini Déterministe (AFD). Le système détecte des séquences d'attaque multi-étapes en temps réel à partir de logs de sécurité bruts transmis via webhook.

L'architecture s'articule autour de deux composants principaux :

- **Workflow n8n** : reçoit les logs JSON, applique la table de transition AFD, persiste les états dans Google Sheets et route les alertes.
- **Système d'alertes multi-canal** : notifie l'analyste SOC simultanément via Slack et WhatsApp (Twilio) dès qu'un état **q3** (Compromis) est atteint.

2 Fondements théoriques : Automate Fini Déterministe

2.1 Définition formelle

L'AFD est défini par le quintuplet $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ où :

Composant	Valeur
Q	$\{q_0, q_1, q_2, q_3\}$
Σ	$\{b, s, p, n, r\}$
q_0	État initial : Calme
F	$\{q_3\}$: état d'acceptation (Compromis)

2.2 États et sémantique

État	Label	Signification
q0	Calme	Aucune activité suspecte
q1	Suspect	Seuil d'échecs d'authentification atteint
q2	Attaque_En_Cours	Scan de ports détecté après phase suspecte
q3	Compromis	Escalade de privilèges — alerte critique déclenchée

2.3 Alphabet et correspondance événements

Sym.	Événement(s)	Condition
b	auth_fail	$\geq N_{auth} = 3$ échecs dans la fenêtre W_{auth}
n	auth_fail	$< N_{auth}$ (non significatif)
s	port_scan, nmap_scan	—
p	privilege_escalation, sudo, su_root	—
r	alert_ack, reset	Acquittement explicite SOC

2.4 Table de transition δ

$\delta(q, \sigma)$	b	s	p	n	r
q0	q1	q0	q0	q0	q0
q1	q1	q2	q1	q1	q1
q2	q2	q2	q3	q2	q2
q3	q3	q3	q3	q3	q0

La seule transition permettant de quitter **q3** est $\delta(q_3, r) = q_0$, déclenchée par l'acquittement explicite de l'analyste SOC.

2.5 Paramètres temporels

Paramètre	Valeur	Rôle
N_{auth}	3 échecs	Seuil de basculement vers q1
W_{auth}	5 min	Fenêtre glissante pour auth_fail
T_{reset}	15 min	Inactivité $\Rightarrow$ retour automatique à q0

3 Architecture du workflow n8n

3.1 Vue d'ensemble

Le workflow est déployé sur `arwa91.app.n8n.cloud` et orchestré en dix nœuds interconnectés formant un pipeline de traitement temps réel.

3.2 Description des nœuds

Nœud	Type	Rôle
Webhook — Réception Logs	Webhook Trigger	Point d'entrée POST JSON
Get row(s) in sheet	Google Sheets (Read)	Lecture état AFD précédent
Code — Moteur AFD	Code Node (JS)	Applique $\delta(q, \sigma)$
Append row in sheet	Google Sheets (Write)	Sauvegarde du nouvel état
IF — Alerte q3 ?	IF Node	Détecte is_alert = true
Switch — État AFD	Switch Node	Route selon l'état courant
HTTP Request (Slack)	HTTP Request	Alerte canal #alerts-siem
HTTP Request1 (WhatsApp)	HTTP Request	Alerte Twilio WhatsApp
Code — Reset q3 $\rightarrow$ q0	Code Node (JS)	Acquittement SOC
Webhook Response	Respond to Webhook	Retourne l'état AFD JSON

TABLE 1 – Les 10 nœuds du workflow n8n

3.3 Nœud déclencheur Webhook

Chaque log de sécurité est transmis via une requête `POST` au webhook. Le corps attendu est un objet JSON minimal contenant obligatoirement `ip_source` et `event_type`.

```
1 {
2   "ip_source": "192.168.1.100",
3   "event_type": "auth_fail"
4 }
```

Listing 1 – Format du log JSON transmis au webhook

3.4 Nœuds IF et Switch

IF — Alerte q3 ? Évalue `is_alert`. Si `true`, les deux nœuds HTTP Request (Slack + Twilio) sont activés en parallèle. Sinon, le flux passe au Switch.

Switch — État AFD Route selon l'état résultant :

- Branche **q1** — Suspect : surveillance renforcée
- Branche **q2** — Attaque_En_Cours : log critique
- Branche **q0** — Calme : traitement standard

3.5 Webhook Response

```
1 {
2   "status": "processed",
3   "state": "q3",
4   "ip": "192.168.1.100",
5   "is_alert": true,
6   "label": "Compromis",
7   "timestamp": "2026-05-03T13:00:00.373Z"
8 }
```

Listing 2 – Réponse JSON confirmant l'état AFD courant

4 Persistance des états : Google Sheets SIEM-State-Store

n8n ne maintient aucune mémoire entre deux exécutions. La persistance de l'état AFD est assurée par une feuille Google Sheets utilisée comme base de données externe : à chaque exécution, l'état courant est lu puis mis à jour.

4.1 Structure de la feuille

Colonne	Description
ip_source	Adresse IP surveillée
event_type	Type d'événement reçu
symbol	Symbole AFD mappé (b, s, p, n, r)
state_before	État AFD avant la transition
state_after	État AFD après la transition
current_label	Label (Calme, Suspect, Attaque_En_Cours, Compromis)
auth_fail_count	Nombre d'échecs dans la fenêtre temporelle
is_alert	TRUE si q3 est atteint pour la première fois
timestamp	Horodatage ISO 8601

4.2 Capture réelle du SIEM-State-Store

La figure ci-dessous montre les données effectivement enregistrées lors de l'exécution du scénario de test complet pour l'IP 192.168.1.100.

	A	B	C	D	E	F	G	H	I	J	K	L
1	ip_source	event_type	symbol	state_before	state_after	current_label	auth_fail_count	is_alert	timestamp			
2	192.168.1.100	auth_fail	n	q0	q0	Calme	1	FALSE	2026-05-03T13:38:15.847Z			
3	192.168.1.100	auth_fail	n	q0	q0	Calme	2	FALSE	2026-05-03T13:38:44.259Z			
4	192.168.1.100	auth_fail	b	q0	q1	Suspect	3	FALSE	2026-05-03T13:39:12.234Z			
5	192.168.1.100	port_scan	s	q1	q2	Attaque_En_Cours	0	FALSE	2026-05-03T13:39:50.531Z			
6	192.168.1.100	privilege_escalat	p	q2	q3	Compromis	0	TRUE	2026-05-03T13:40:26.192Z			
7	192.168.1.100	alert_ack	r	q3	q0	Calme	0	FALSE	2026-05-03T13:41:09.786Z			
8												
9												
10												
11												
12												
13												
14												
15												
16												
17												
18												
19												
20												
21												

FIGURE 1 – Google Sheets SIEM-State-Store — données réelles enregistrées lors des tests

La séquence enregistrée illustre la progression complète **q0**→**q1**→**q2**→**q3** :

#	event_type	sym	avant	après	label	alerte
1	auth_fail	n	q0	q0	Calme	FALSE
2	auth_fail	n	q0	q0	Calme	FALSE
3	auth_fail	b	q0	q1	Suspect	FALSE
4	port_scan	s	q1	q2	Attaque_En_Cours	FALSE
5	privilege_escalation	p	q2	q3	Compromis	TRUE
6	alert_ack	r	q3	q0	Calme	FALSE

5 Moteur AFD : Code Node JavaScript

5.1 Table de transition implémentée

```

1  const PARAMS = { Nauth: 3, Wauth: 5*60*1000, Treset: 15*60*1000 };
2
3  const DELTA = {
4    q0: { b:'q1', s:'q0', p:'q0', n:'q0', r:'q0' },
5    q1: { b:'q1', s:'q2', p:'q1', n:'q1', r:'q1' },
6    q2: { b:'q2', s:'q2', p:'q3', n:'q2', r:'q2' },
7    q3: { b:'q3', s:'q3', p:'q3', n:'q3', r:'q0' },
8  };
9
10 const SYMBOL_MAP = {
11   auth_fail:      (c) => c >= PARAMS.Nauth ? 'b' : 'n',
12   port_scan:      () => 's',
13   nmap_scan:      () => 's',
14   privilege_escalation: () => 'p',
15   sudo:           () => 'p',
16   su_root:        () => 'p',
17   alert_ack:      () => 'r',
18   reset:          () => 'r',
19 };

```

Listing 3 – Table de transition δ et mapping événements — Code Node

5.2 Logique principale avec fenêtres temporelles

```

1  // Recuperation du dernier etat valide depuis Google Sheets
2  const validRows = $('Get row(s) in sheet').all()
3    .filter(r => r.json && r.json.ip_source === ip
4      && r.json.state_after?.trim() !== '');
5  const lastRow = validRows[validRows.length - 1]?.json;
6  let currentState = lastRow?.state_after || 'q0';
7
8  // Timeout T_reset : inactivite > 15 min -> retour q0
9  if ((NOW - stateEnteredAt) > PARAMS.Treset
10    && currentState !== 'q0' && currentState !== 'q3') {
11    currentState = 'q0';
12    authFailCount = 0;
13  }
14

```



```

15 // Fenetre glissante W_auth : conservation des auth_fail < 5 min
16 if (eventType === 'auth_fail') {
17   authFailTimes.push(NOW);
18   authFailTimes = authFailTimes.filter(t => (NOW - t) < PARAMS.
      Wauth);
19   symbol = authFailTimes.length >= PARAMS.Nauth ? 'b' : 'n';
20 }
21
22 // Application de la transition delta
23 const nextState = DELTA[currentState]?.[symbol] || currentState;
24 const isAlert    = (nextState === 'q3' && currentState !== 'q3');

```

Listing 4 – Extrait — gestion de l'état, fenêtres temporelles et transition

6 Système d'alertes multi-canal

Lorsque le nœud *IF* — *Alerte q3?* évalue `is_alert = true`, deux nœuds HTTP Request sont activés simultanément pour notifier l'analyste SOC.

6.1 Alerte Slack — canal #alerts-siem

6.1.1 Configuration technique

Paramètre	Valeur
Méthode	POST
URL	<code>https://hooks.slack.com/services/TOB1N8GPDDX/[token]</code>
Content-Type	<code>application/json</code>
Canal cible	<code>#alerts-siem</code> (workspace : projects)
Authentication	Token intégré dans l'URL Incoming Webhook

6.1.2 Payload JSON envoyé

```

1 {
2   "text": ":rotating_light: SIEM ALERT: COMPROMIS DETECTE :
      rotating_light:
3   IP: {{ $json.ip_source }}
4   Transition: q2 -> q3
5   Label: Compromis"
6 }

```

Listing 5 – Message Slack déclenché lors d'une transition vers q3

6.1.3 Preuve de réception — canal Slack

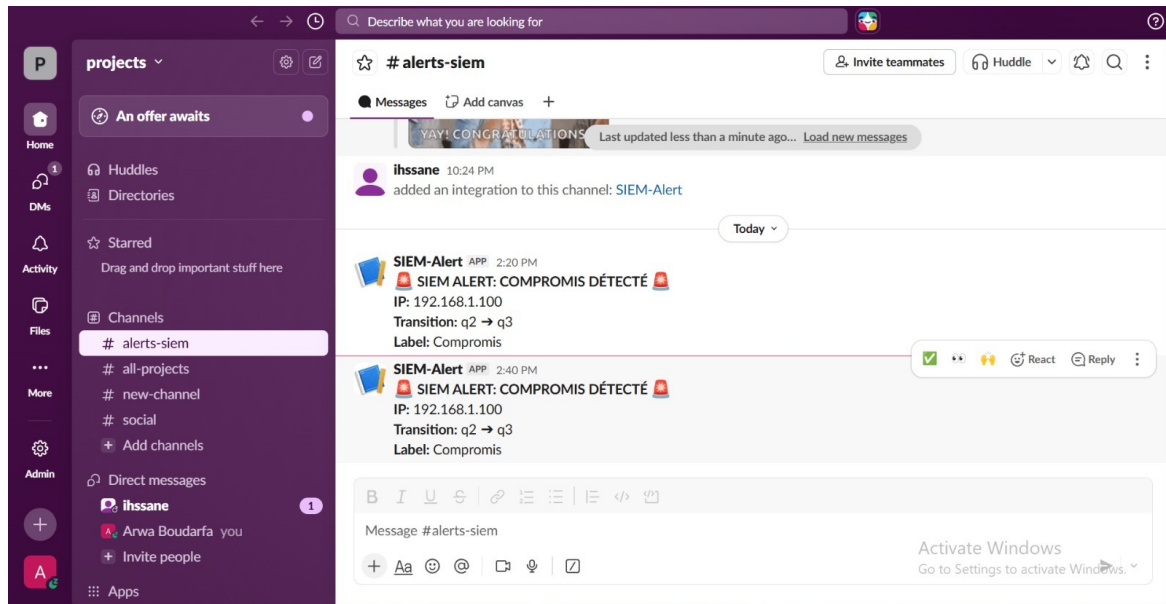


FIGURE 2 – Canal Slack #alerts-siem — alertes SIEM reçues en temps réel (workspace : projects)

La capture confirme la réception de deux alertes consécutives avec le message *SIEM ALERT : COMPROMIS DÉTECTÉ*, l'IP source, la transition **q2→q3** et le label *Compromis*.

6.2 Alerte WhatsApp — API Twilio Sandbox

6.2.1 Présentation de Twilio

Twilio est une plateforme CPaaS (Communications Platform as a Service) qui expose des APIs REST pour l'envoi de messages WhatsApp, SMS et appels vocaux. Le *Twilio Sandbox for WhatsApp* permet d'envoyer des messages réels via le numéro +14155238886 sans nécessiter d'approbation complète par Meta.

6.2.2 Activation du sandbox

1. Créer un compte sur <https://www.twilio.com>
2. Aller dans **Messaging** → **Try it out** → **Send a WhatsApp message**
3. Envoyer `join come-steady` au numéro +14155238886 depuis WhatsApp
4. Confirmer la réception du message Twilio : *"You are all set!"*
5. Récupérer l'Account SID et l'Auth Token dans la console Twilio

6.2.3 Configuration du nœud HTTP Request (WhatsApp)

Paramètre	Valeur
Méthode	POST
URL	https://api.twilio.com/2010-04-01/Accounts/AC981a.../Messages
Authentication	HTTP Basic Auth (Account SID + Auth Token)
Content-Type	application/x-www-form-urlencoded
From	whatsapp:+14155238886
To	whatsapp:+212620780771

L'Auth Token est géré via les *Credentials* sécurisés de n8n et n'apparaît jamais en clair dans le JSON exporté du workflow.

6.2.4 Réponse API Twilio confirmée

```

1  {
2    "status":          "queued",
3    "to":              "whatsapp:+212620780771",
4    "from":            "whatsapp:+14155238886",
5    "body":            "ALERTE SOC - IP Compromise ! Etat AFD : q3
                        COMPROMIS...",
6    "error_code":      null,
7    "error_message":   null
8  }

```

Listing 6 – Réponse Twilio — statut queued (succès)

`status: "queued"` et `error_code: null` confirment que le message a été accepté par l'API Twilio et mis en file de livraison WhatsApp.

6.2.5 Preuve de réception WhatsApp

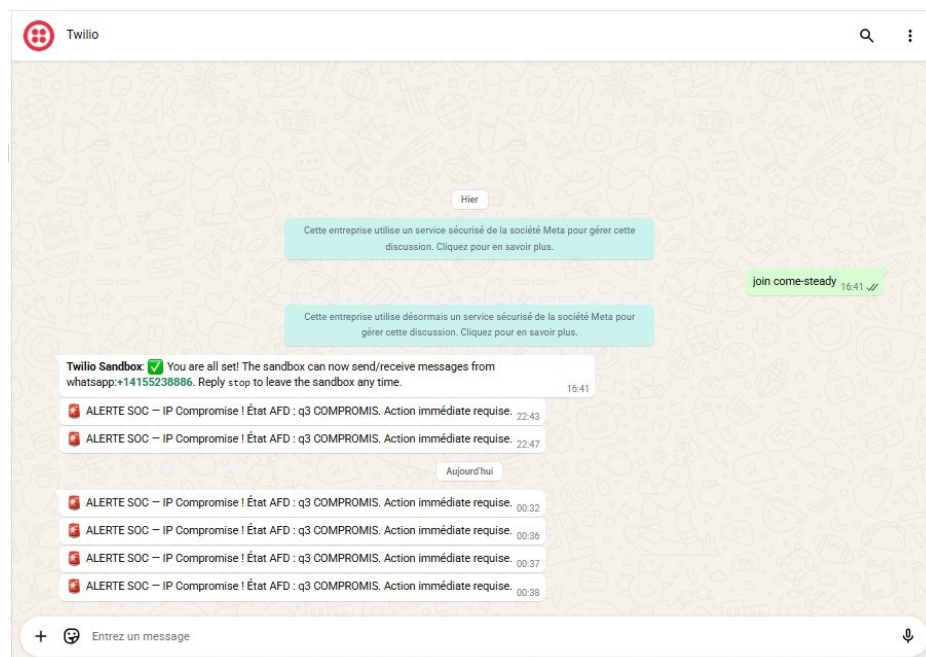


FIGURE 3 – WhatsApp — alertes SOC reçues sur le numéro analyste depuis le Sandbox Twilio (+14155238886)

La capture montre la séquence complète : activation du sandbox ("*join come-steady*" à 16 :41), confirmation Twilio ("*You are all set!*"), puis six alertes reçues — *ALERTE SOC — IP Compromise! État AFD : q3 COMPROMIS. Action immédiate requise.* — aux horaires 22 :43, 22 :47 (hier) et 00 :32, 00 :36, 00 :37, 00 :38 (aujourd'hui). Ces six messages correspondent aux tests successifs de la séquence d'attaque.

7 Tests et validation

7.1 Outil de test : Postman

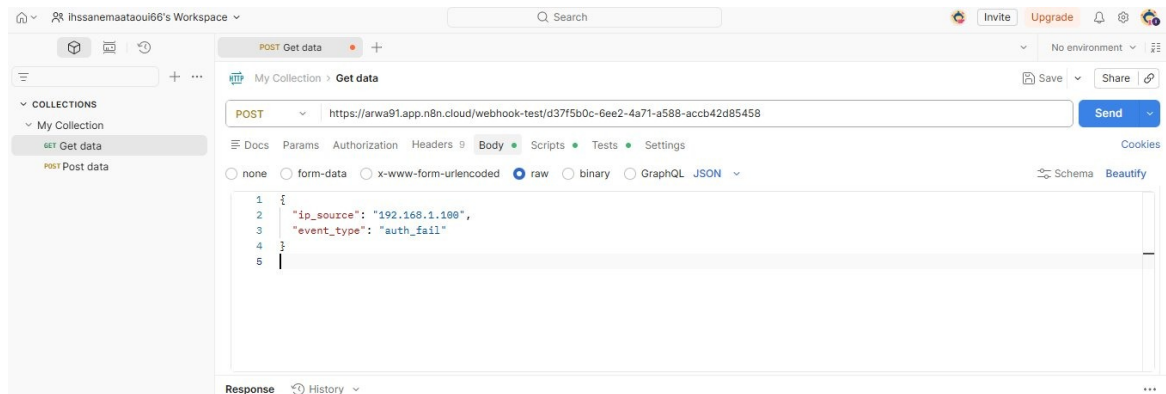


FIGURE 4 – Postman — envoi d'une requête POST au webhook avec le log JSON

7.2 Séquence d'attaque testée

La validation complète repose sur trois requêtes envoyées dans l'ordre suivant depuis Postman, reproduisant un scénario d'attaque réaliste :

Requête 1 — Échecs d'authentification répétés (envoyée 3 fois)

```
1 {
2   "ip_source": "192.168.1.100",
3   "event_type": "auth_fail"
4 }
```

Listing 7 – Log auth_fail — déclenche la transition $q_0 \rightarrow q_1$

Requête 2 — Scan de ports

```
1 {
2   "ip_source": "192.168.1.100",
3   "event_type": "port_scan"
4 }
```

Listing 8 – Log port_scan — déclenche la transition $q_1 \rightarrow q_2$

Requête 3 — Escalade de privilèges

```
1 {
2   "ip_source": "192.168.1.100",
3   "event_type": "privilege_escalation"
4 }
```

Listing 9 – Log privilege_escalation — déclenche $q_2 \rightarrow q_3$ et l'alerte

7.3 Résultats étape par étape

Étape	event_type	sym	avant	après	Résultat
1	auth_fail	n	q0	q0	count = 1
2	auth_fail	n	q0	q0	count = 2
3	auth_fail	b	q0	q1	Suspect
4	port_scan	s	q1	q2	Attaque_En_Cours
5	privilege_escalation	p	q2	q3	ALERTE ENVOYÉE

7.4 Tableau de validation des fonctionnalités

Fonctionnalité	Statut	Preuve
Webhook Trigger — réception logs JSON	Validé	Tests Postman
Code Node — Moteur AFD + table δ	Validé	Code JS fonctionnel
Compteurs temporels ($T_{reset} = 15$ min)	Validé	Timeout intégré
Google Sheets — persistance des états	Validé	Capture Fig. 1
Noeuds IF/Switch pour les transitions	Validé	Workflow JSON exporté
WhatsApp via Twilio Sandbox	Validé	Capture Fig. 3
Alerte Slack via Incoming Webhook	Validé	Capture Fig. 2
Reset q3 → q0 (ack SOC)	Validé	Noeud Code Reset
Scenario complet teste (3 requetes)	Validé	Section 7.2

8 Difficultés rencontrées et solutions

8.1 Persistance d'état entre exécutions

n8n Cloud ne maintient aucune mémoire entre les exécutions : `$workflow.staticData` s'est révélé non persistant. La solution adoptée est l'utilisation de Google Sheets comme base de données d'état externe. Le contexte AFD complet est lu et écrit à chaque exécution.

8.2 Variabilité du format d'entrée

Selon le mode d'exécution (Test vs Production), le JSON entrant est encapsulé dans différentes couches. La solution est une extraction robuste en cascade dans le Code Node :

```

1 const inputData = $('Webhook -- Reception Logs').first();
2 let log = inputData.json || inputData.body || inputData;
3 if (log?.body && typeof log.body === 'object') log = log.body;
4 if (typeof log === 'string') { try { log = JSON.parse(log); } catch
  (e){} }

```

Listing 10 – Extraction résiliente du JSON entrant

8.3 Authentication Twilio dans n8n

La configuration Basic Auth pour Twilio requiert de distinguer Account SID (username) et Auth Token (password). La solution est la création d'un Credential de type `httpBasicAuth` géré de façon sécurisée via le vault n8n — l'Auth Token n'apparaît jamais en clair dans le JSON exporté.

8.4 Mode Test vs Production

En mode test, le webhook n8n n'accepte qu'un seul appel à la fois. La validation finale a été effectuée en mode production après publication du workflow via le bouton **Publish**.

9 Limites identifiées et perspectives

1. **Scalabilité** : Google Sheets devient lent au-delà de quelques centaines d'entrées. Une base Redis ou PostgreSQL serait préférable en production.
2. **Déduplication des alertes** : des tests rapprochés peuvent générer plusieurs alertes **q3**. Un mécanisme de déduplication basé sur un TTL serait nécessaire.
3. **Sandbox Twilio** : limité à des destinataires pré-approuvés. Un compte WhatsApp Business approuvé par Meta est requis pour un déploiement réel.
4. **Fenêtre W_{auth}** : l'efficacité de la fenêtre glissante dépend de la fréquence d'exécution et de la latence Google Sheets.
5. **Multi-IP à forte charge** : la lecture de Google Sheets est séquentielle, ce qui peut créer des goulots d'étranglement.

10 Conclusion

Ce projet démontre l'application concrète et opérationnelle des Automates Finis Déterministes à un problème réel de cybersécurité. Le mini-SIEM CY-01 est entièrement fonctionnel : il reçoit des logs via webhook, applique la table de transition AFD, persiste les états dans Google Sheets, et déclenche des alertes simultanées vers Slack et WhatsApp dès la détection d'un compromis.

Résultats validés :

- Séquence d'attaque complète (3 types d'événements, 5 étapes) détectée et tracée
- Alertes WhatsApp reçues sur le numéro analyste SOC via Twilio Sandbox
- Alertes Slack postées en temps réel dans le canal **#alerts-siem**
- États AFD correctement persistés dans Google Sheets à chaque transition
- Reset **q3→q0** opérationnel (acquittement SOC)

Démonstration complète — Google Drive

<https://drive.google.com/drive/u/0/folders/1Io27xpcMtYtqF5CFyB3RTVWAKiHw8BrE>

Workflow JSON, captures, exports et vidéo de démonstration

Arwa Boudarfa | Ihssane El Maataoui

ENSAM Casablanca — Cybersécurité & Cloud Computing — 2025-2026